

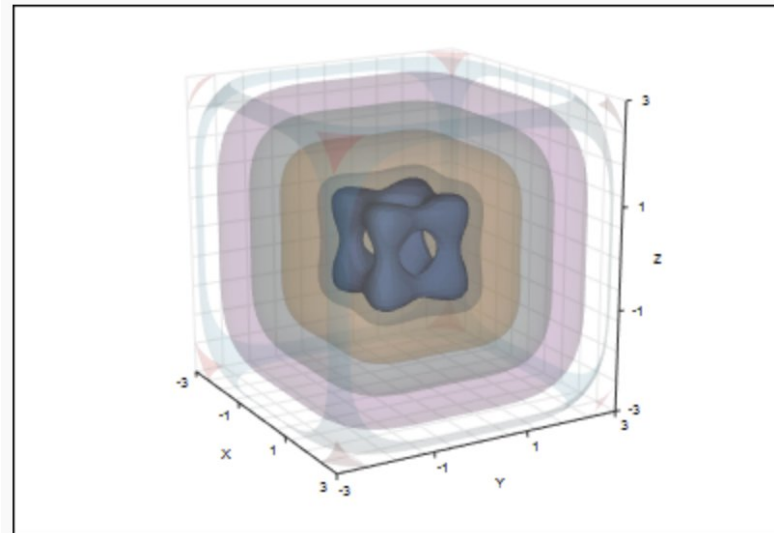


IsoSurfaces

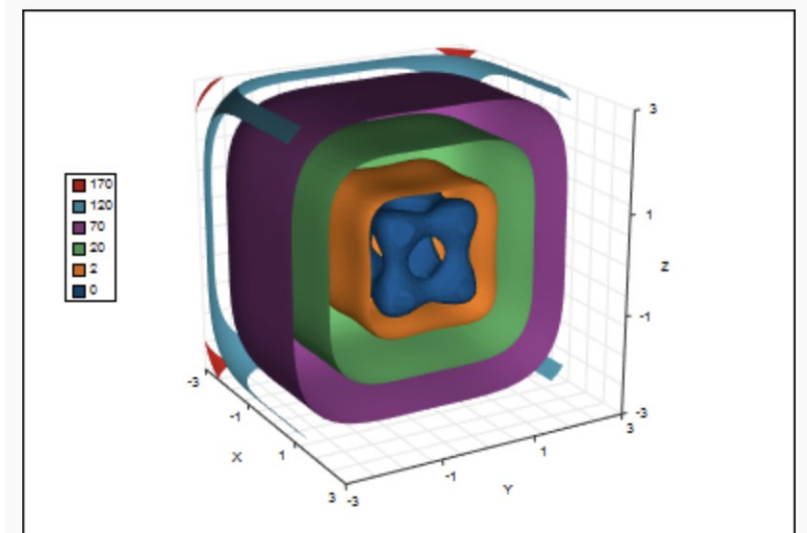
Luís Paulo Santos

*Departamento de
Informática*

2025



Isosurfaces of the function $w = x^4 + y^4 + z^4 - (x^2 + y^2 + z^2 - 0.4)$. The 0 level isosurface has been plotted solid, with the others plotted transparent.

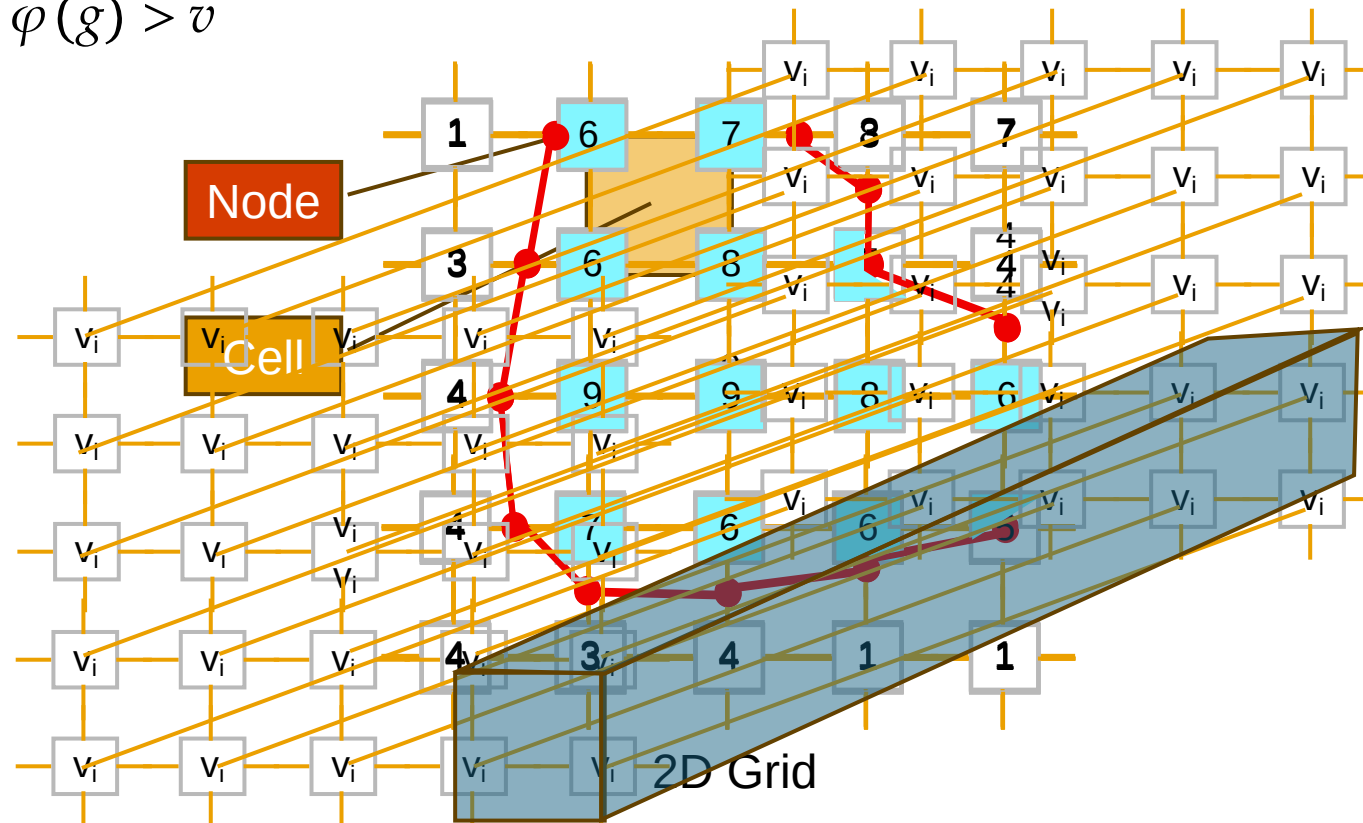


The same function as the transparent plot above, this time using slightly different isosurface values, and plotted over a limited range in the x direction to give a "cut-away" effect.



IsoSurfaces: Concept

- Build a surface (2d: contour) containing all data points with the same value, v
- An isosurface can be thought of as a boundary between regions of the data field, φ , where $\varphi(g) < v$ and $\varphi(g) > v$



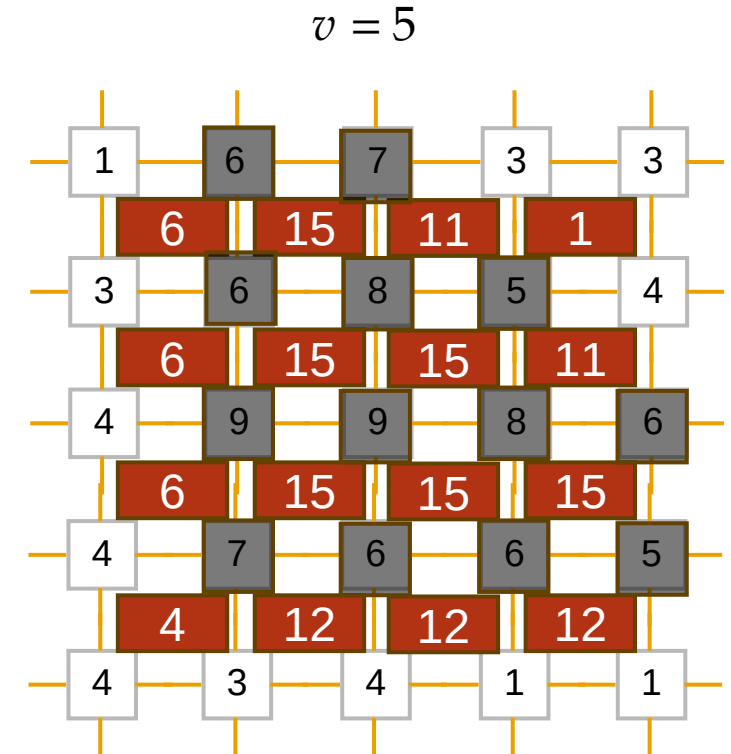
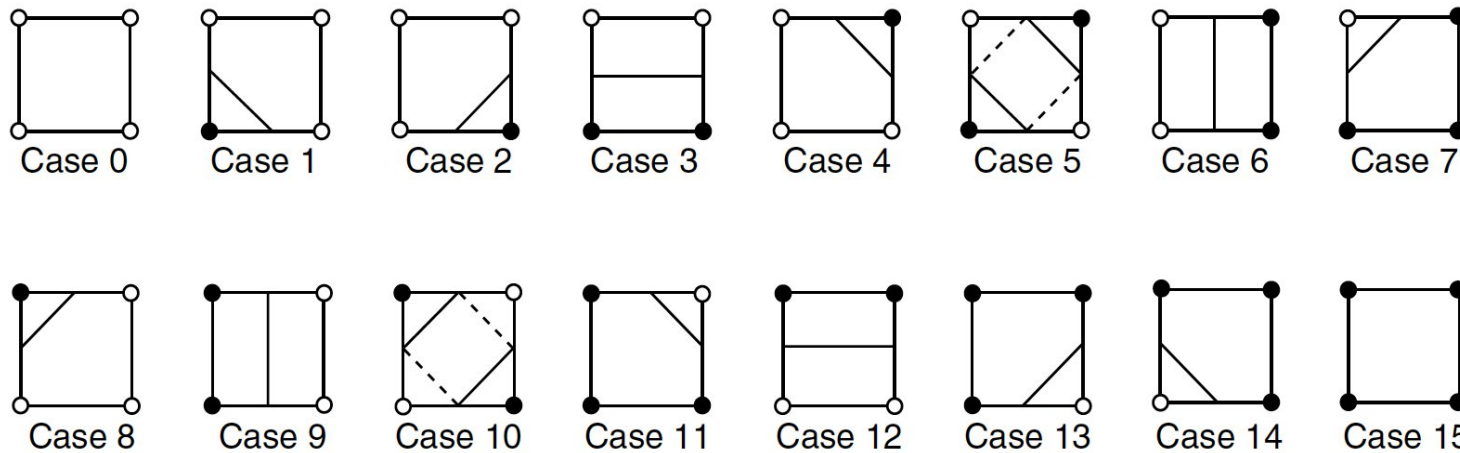


- Let $\varphi: G \rightarrow V$ be a scalar field,
with $G \subseteq \mathbb{R}^d$ be a d-dimensional geometric space and $V \subseteq \mathbb{R}$ be the value space
- Let $\mathcal{D}$ be a sample data set over φ , with $\mathcal{D} = \{d_i\}$ and $d_i \in D = G \times V$,
and $d = |\mathcal{D}|$ be the dataset size
- **Isosurface Extraction:**
Given $\mathcal{D}$ and a single value $v \in V$, find a set $S = \{g_i \mid g_i \in G \text{ and } \varphi(g_i) = v\}$
- **Grid:**
Set $\mathcal{C}$ of cells, where each cell $c \in \mathcal{C}$ is a set of nodes $c = \{n\} \subset \mathcal{D} = G \times V$, with the nodes in each cell exhibiting some notion of neighborhood in G
- Approaches to Isosurface extraction:
 - **geometric decomposition**: directly search the cells in the grid
 - **value decomposition** : search within a structure representing each grid cell by its minimum and maximum values
 - **image space decomposition** : limit the search to grid cells potentially visible from the current view point



Geometric: Marching Squares (2D)

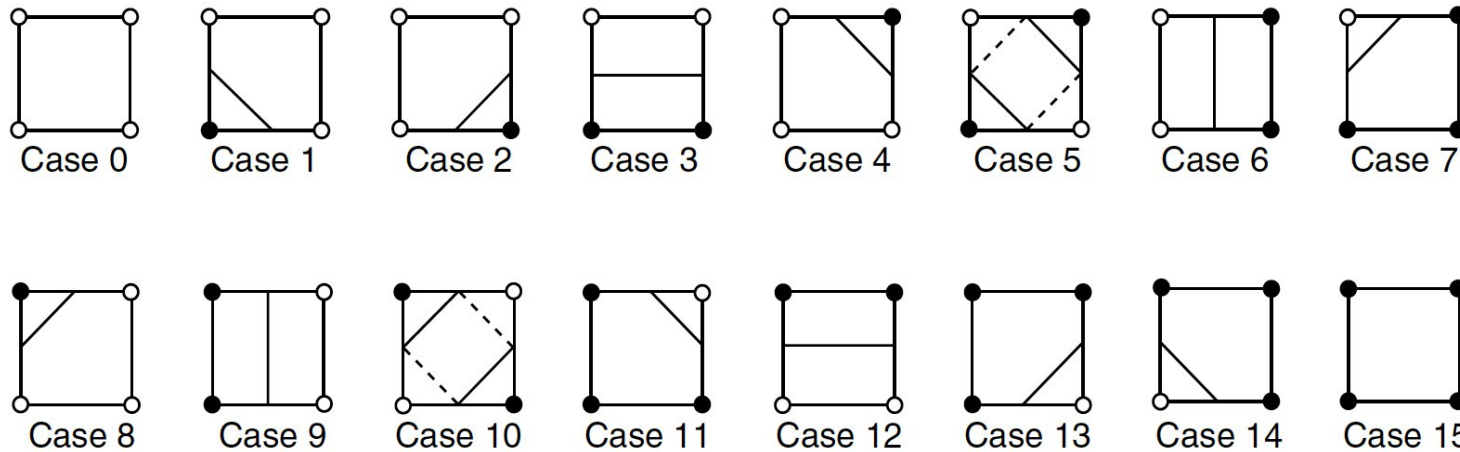
1. Classify each cell's topology, generating a 4-bit index into the cases table





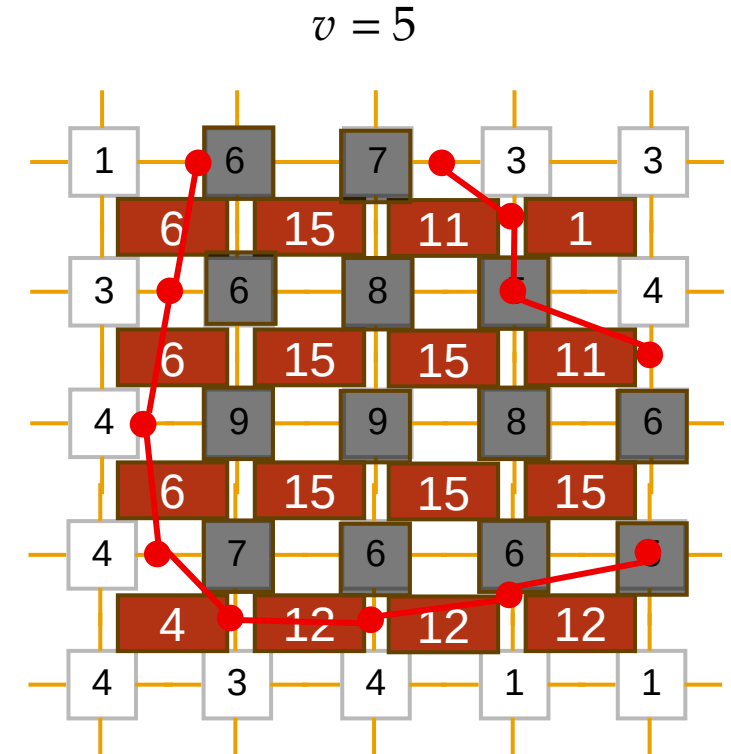
Geometric: Marching Squares (2D)

1. Classify each cell's topology, generating a 4-bit index into the cases table



2. Calculate the contour location (interpolation)
for each edge in the case table

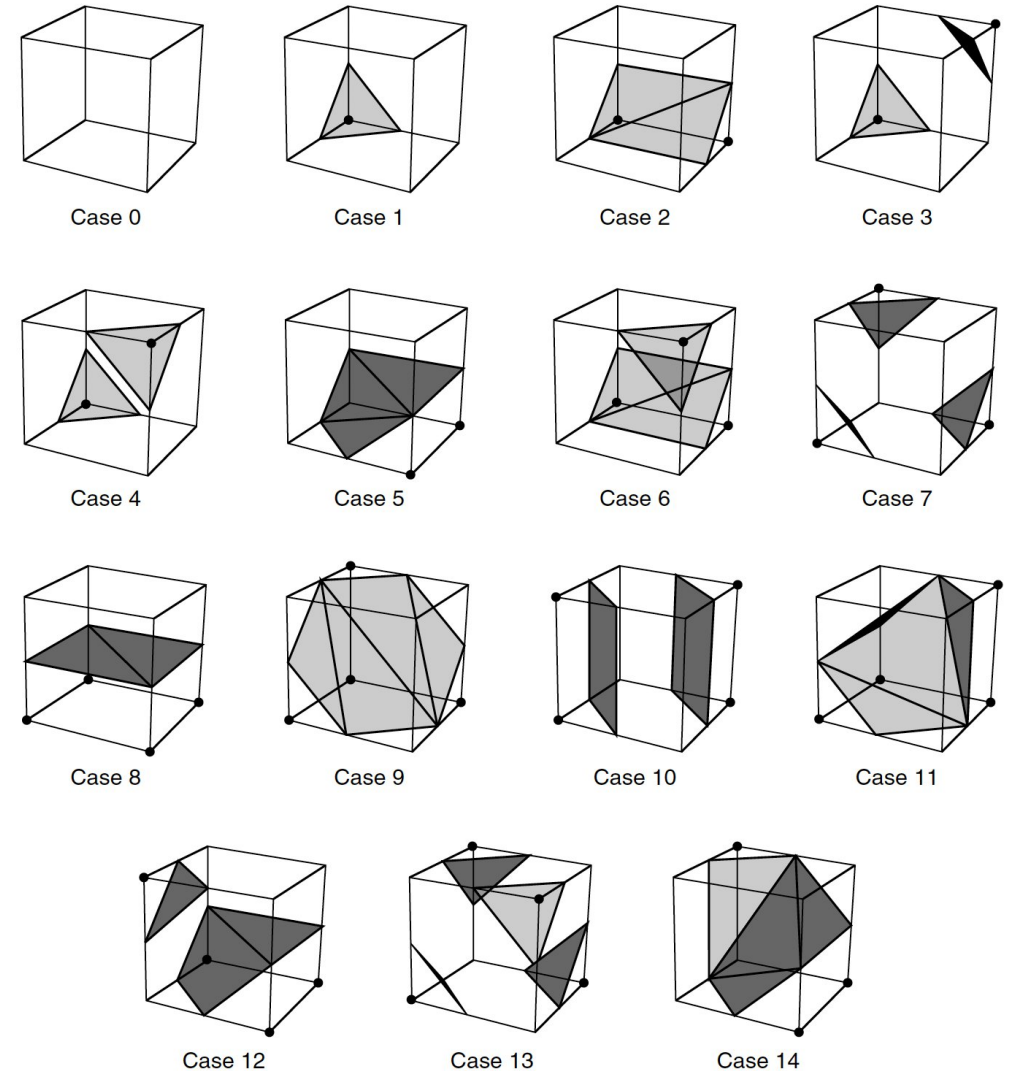
3. Connect contour locations using a geometric primitive
(e.g. straight line)





Geometric: Marching Cubes (3D)

- Rational identical to the 2D case (15 cases by using symmetries)
- Still do interpolation along each relevant edge
- Use triangles to connect lines in the cells faces



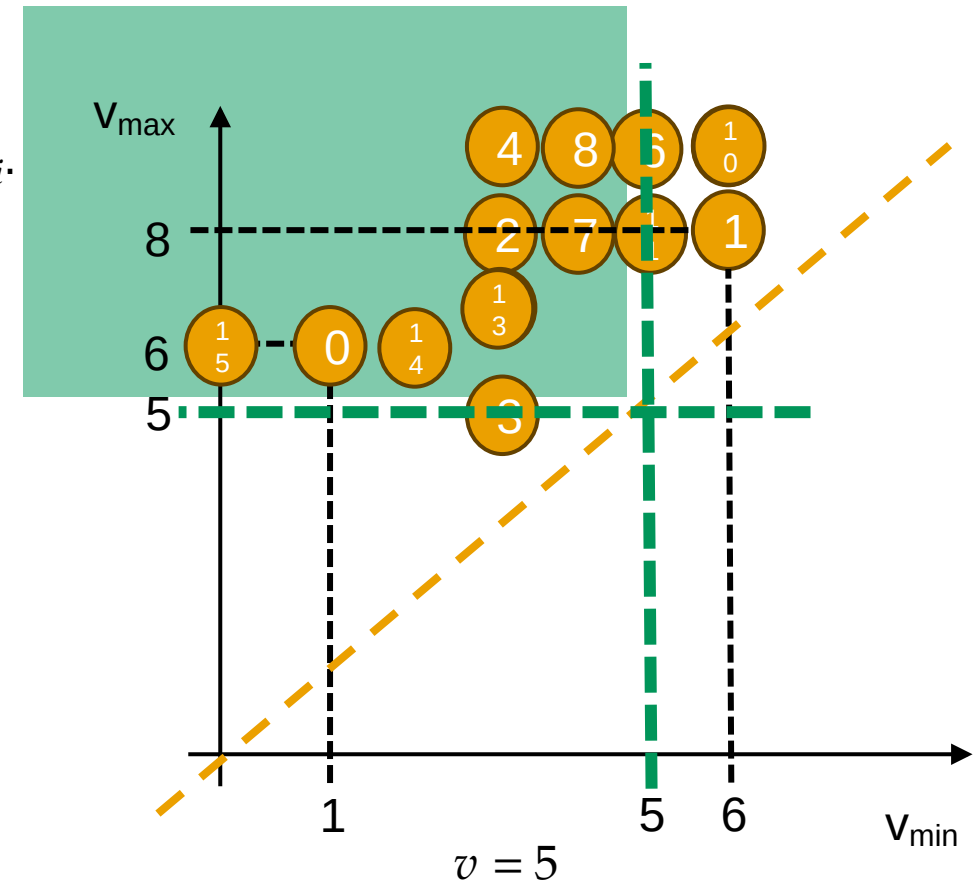
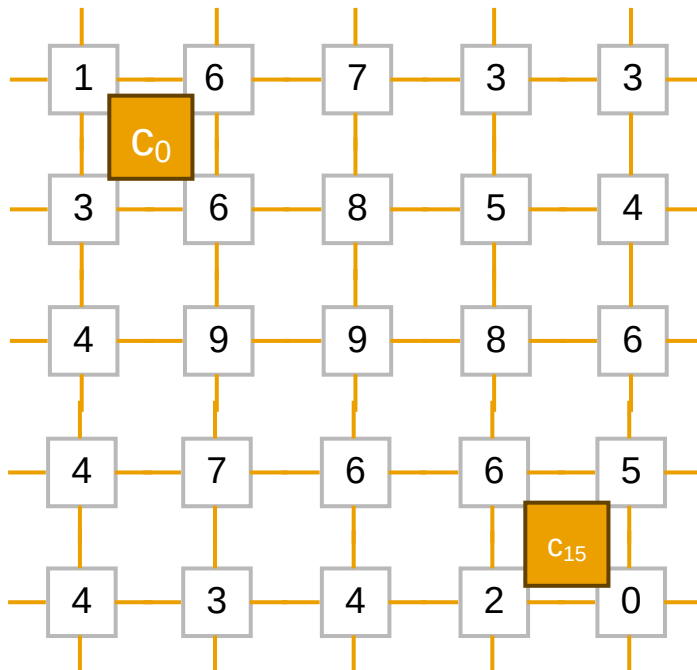
Value decomposition: scan space

- Definition :**

given a set of c cells $\mathcal{C}$ define a set of c points (one per cell) $\mathcal{P} \subset V^2, \mathcal{P} = \{p_i\}$,

such that $p_i = \left(\min_j \left(\{v_j\}_i \right), \max_j \left(\{v_j\}_i \right) \right)$,

and $\{v_j\}_i$ are the values of all nodes belonging to cell c_i .





Near Optimal IsoSurface Extraction (NOISE)

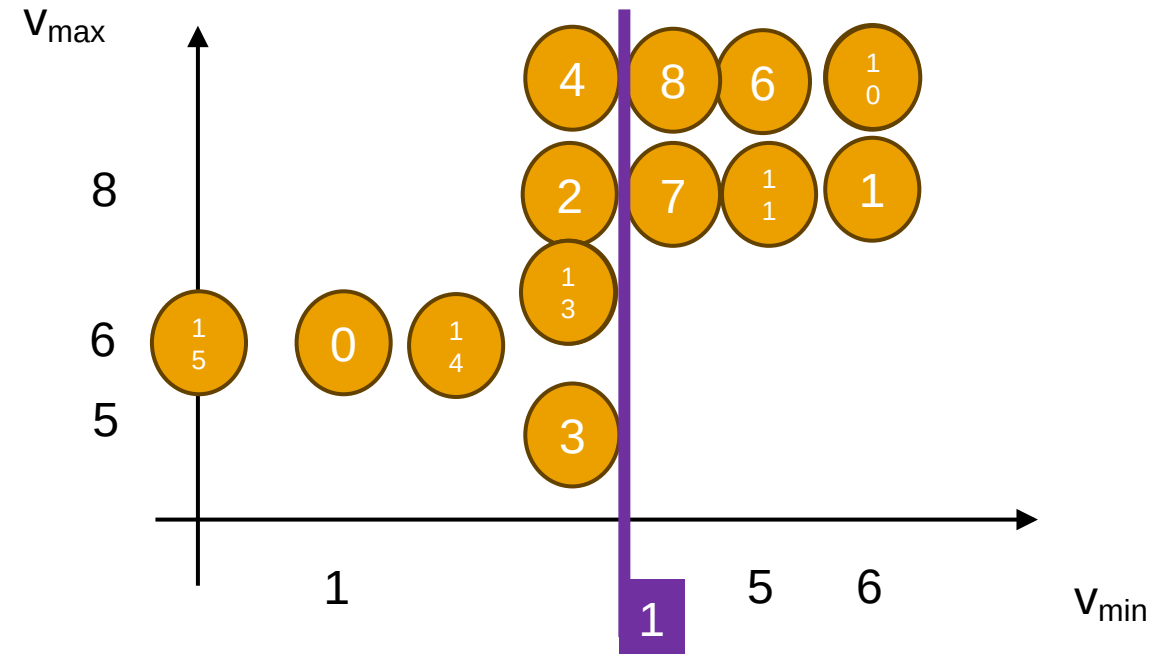
- Build a kd-tree over span space
 1. while a node has more than 1 cell
 1. alternate between dimensions v_{min} and v_{max}
 2. sort along current dimension: v_{min} or v_{max}
 3. split the kd-tree node at the median

4

0,2,3,4
12,13,
14,15

1,5,6,7,
8,9,10,
11

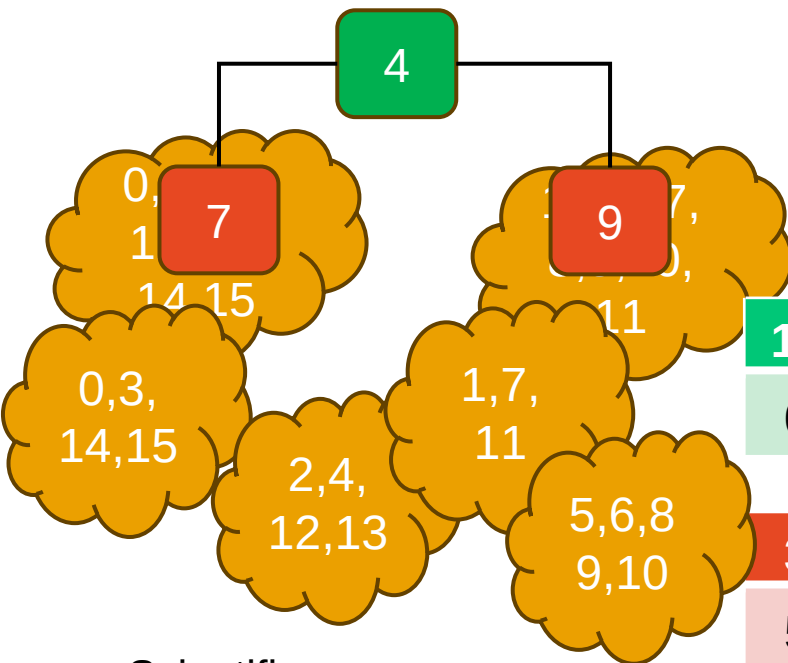
15	0	14	2	3	4	12	13	7	8	6	11	1	5	9	10
0	1	2	3	3	3	3	3	4	4	5	5	6	6	6	6



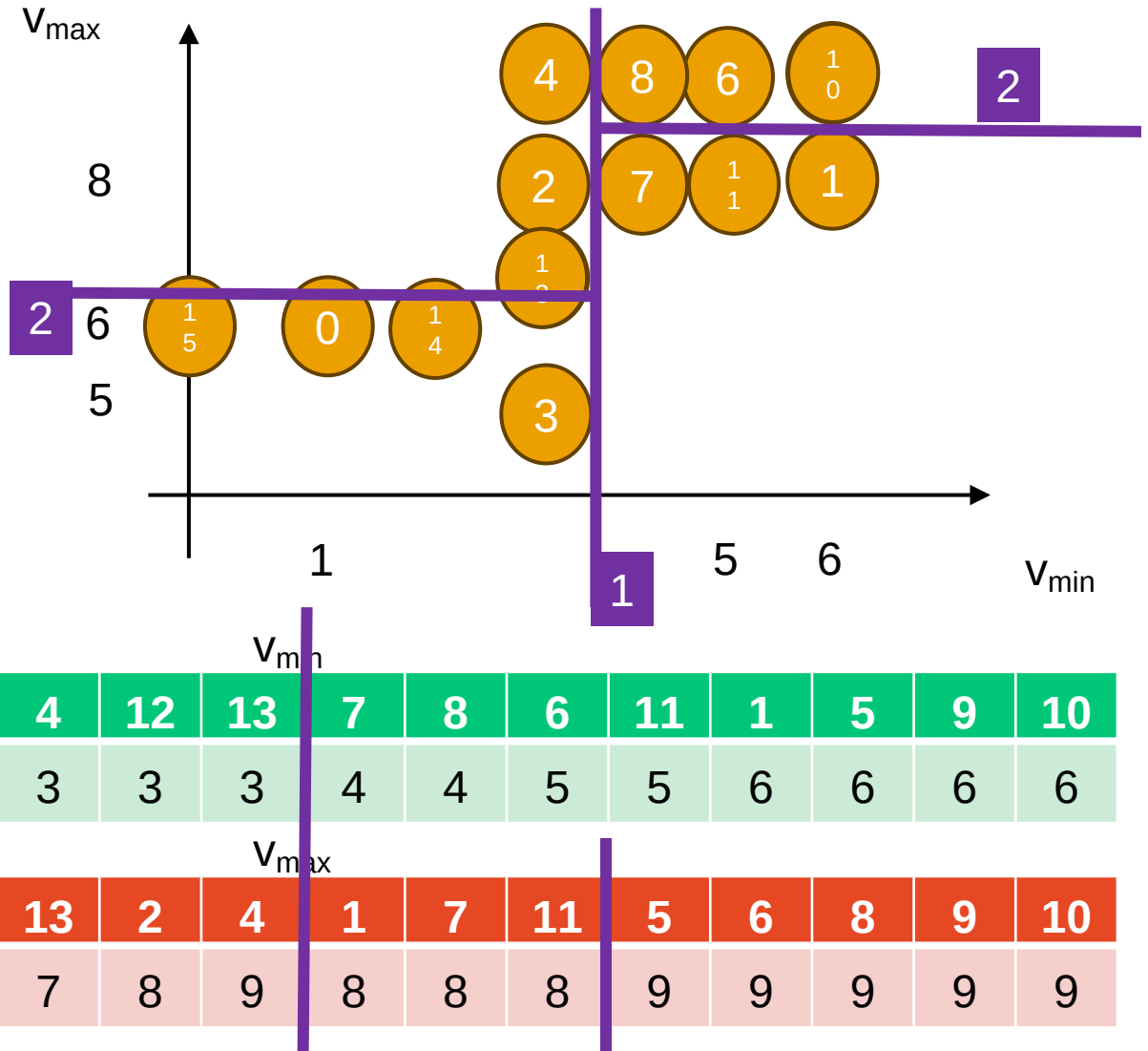


Near Optimal IsoSurface Extraction (NOISE)

- Build a kd-tree over span space
 - while a node has more than 1 cell
 - alternate between dimensions v_{min} and v_{max}
 - sort along current dimension: v_{min} or v_{max}
 - split the kd-tree node at the median



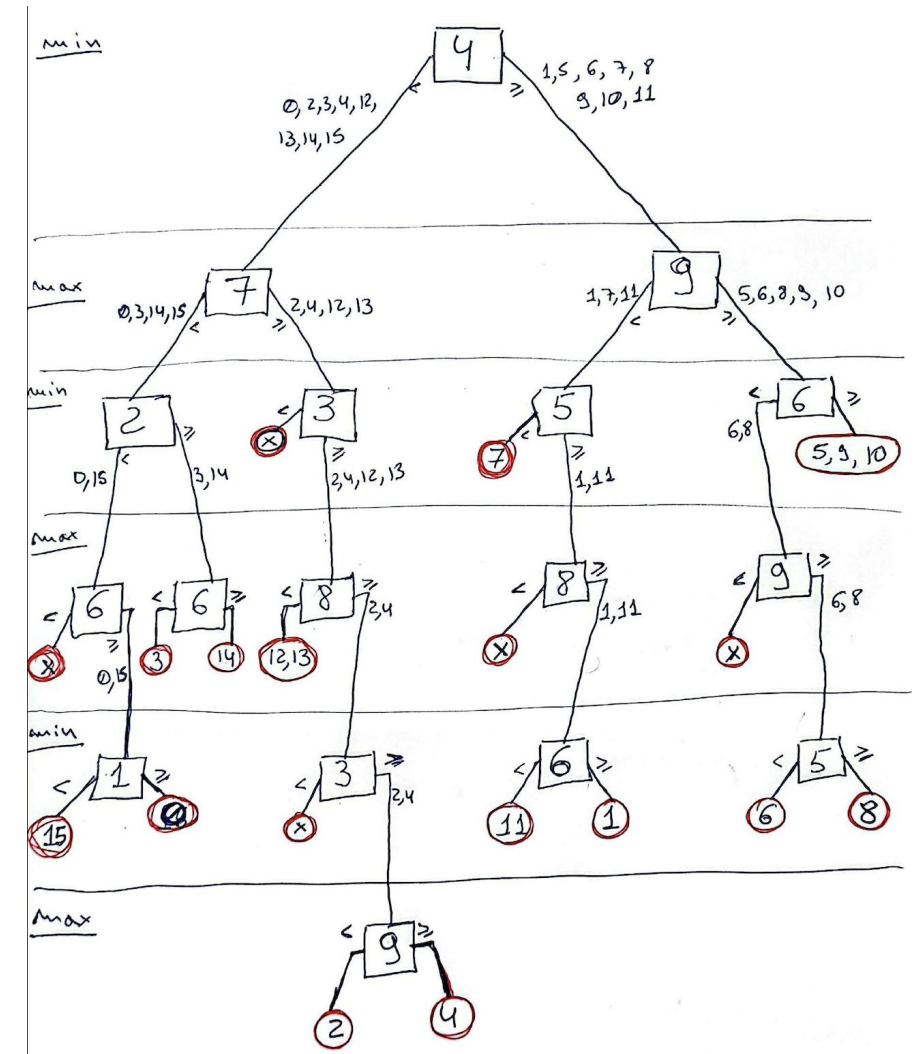
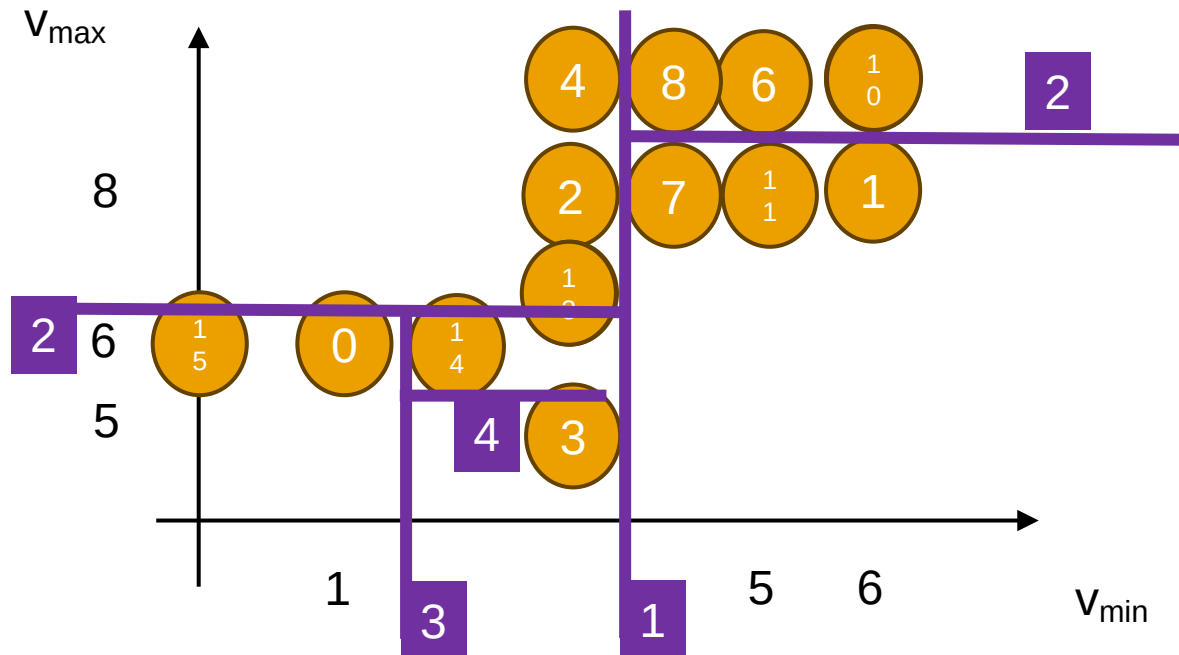
Scientific
Visualization





Near Optimal IsoSurface Extraction (NOISE)

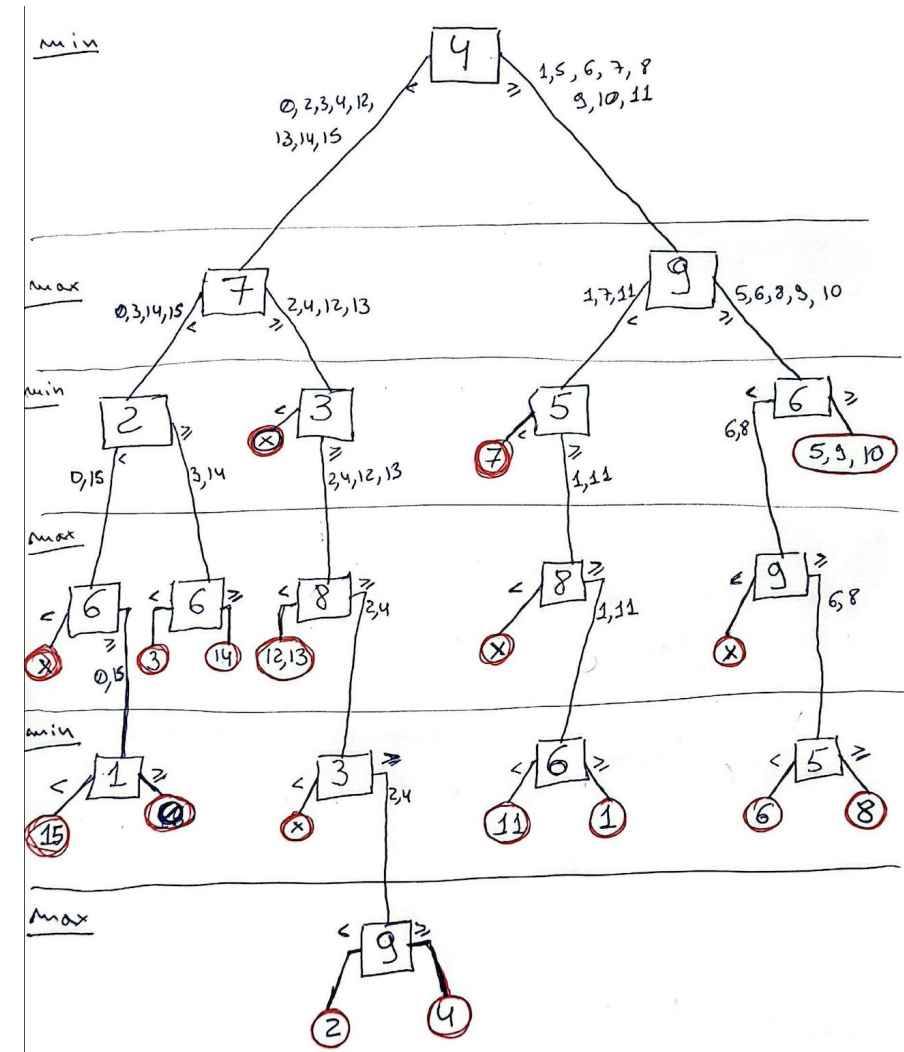
- Build a kd-tree over span space
 - while a node has more than 1 cell
 - alternate between dimensions v_{min} and v_{max}
 - sort along current dimension: v_{min} or v_{max}
 - split the kd-tree node at the median





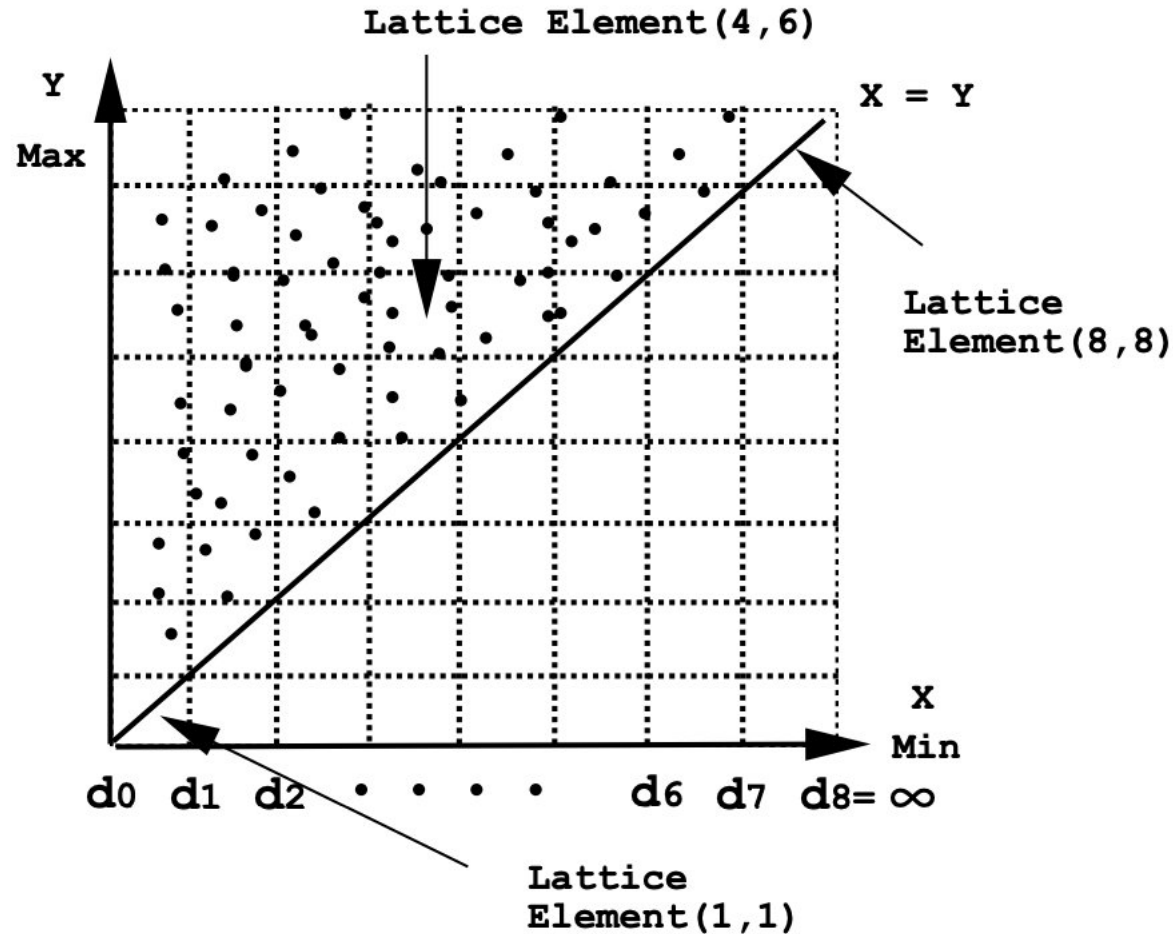
Near Optimal IsoSurface Extraction (NOISE)

- if leaf
 - add cells to surface
 - return
- if current dimension is *min*
 1. recursively search left_branch
 2. if $v \geq v_{node}$
 1. recursively search right_branch
- else if current dimension is *max*
 1. recursively search right_branch
 2. if $v < v_{node}$
 1. recursively search left_branch



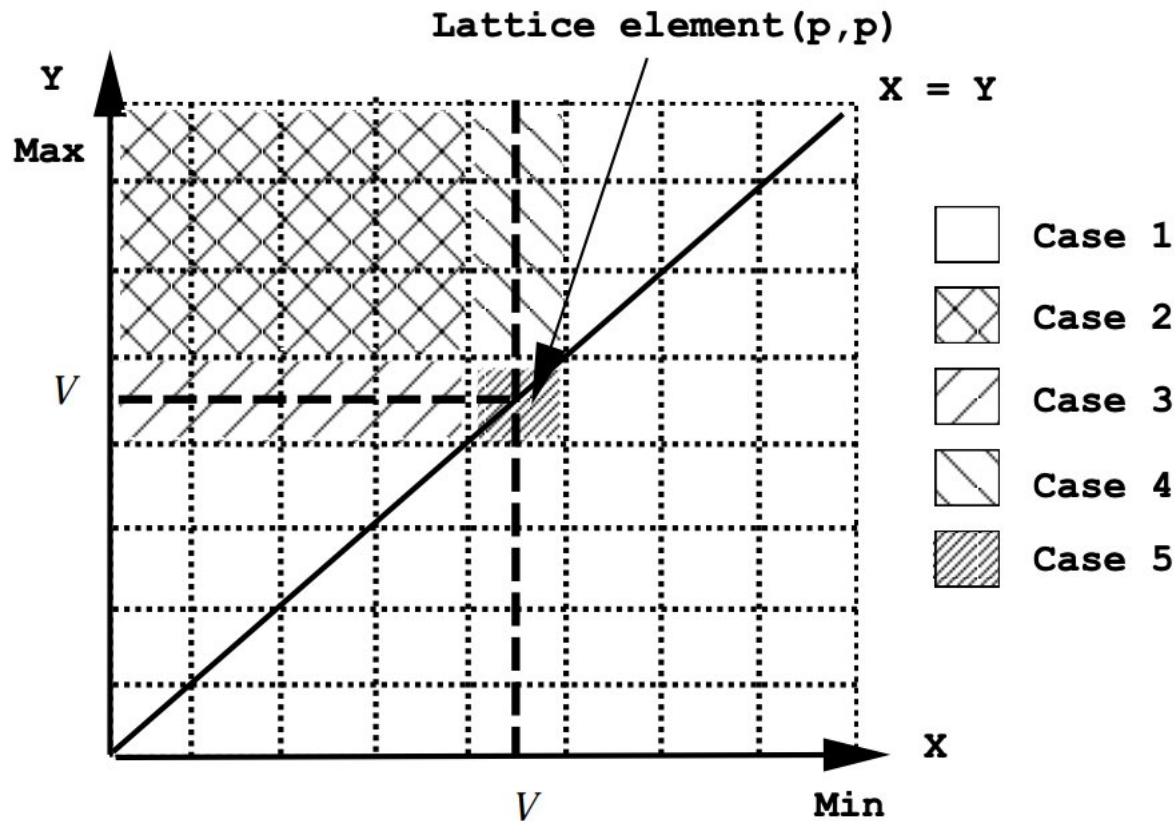


Isosurfacing in Span Space with Utmost Efficiency (ISSUE)





Isosurfacing in Span Space with Utmost Efficiency (ISSUE)



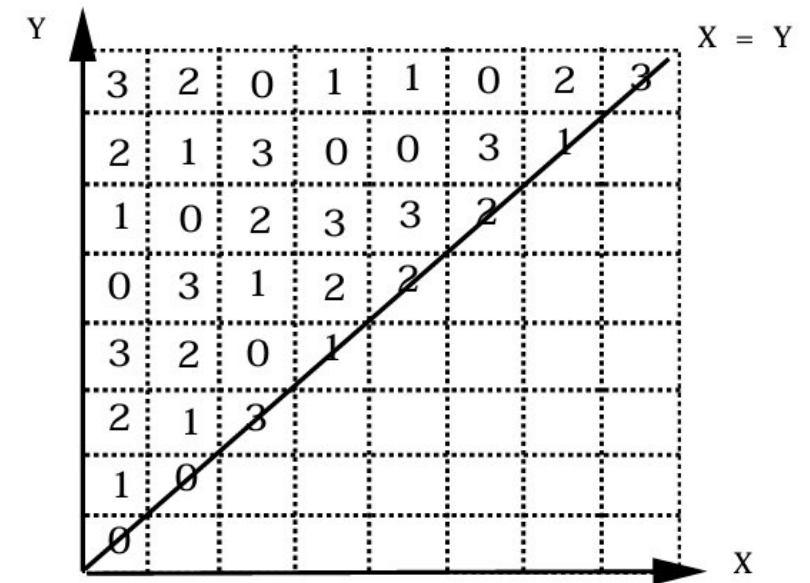
1. $i > p$ or $j < p$: All the cells in this region have either a higher minimum value or lower maximum value than the isovalue. These lattice elements do not contain any isosurface cells.
2. $i < p$ and $j > p$: All the cells in these lattice elements are isosurface cells.
3. $i < p$ and $j = p$: All the cells in this region have a lower minimum value than the isovalue. Hence only those cells that have a higher maximum value than the isovalue are isosurface cells.
4. $i = p$ and $j > p$: All the cells in this region have a higher maximum value than the isovalue. Hence only those cells that have a lower minimum value than the isovalue are isosurface cells.
5. $i = p$ and $j = p$: This is the only lattice element that requires a min-max search to locate the isosurface cells. Any isosurfacing algorithm, such as a Kd-tree searching method or sweeping simplices, will do.



Isosurfacing in Span Space with Utmost Efficiency (ISSUE)

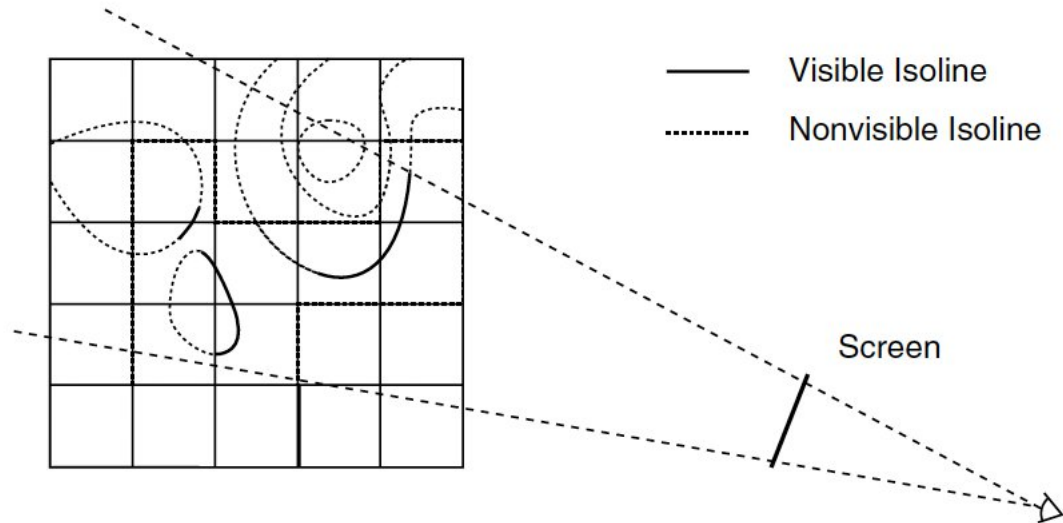
Parallel decomposition and distribution

- cells within any scalar range $[a;b]$ should be evenly scattered, each PE gets the same number of cells for any isovalue.
- unfold the grid elements in the half space above the $X = Y$ line column by column into a 1D list and distribute into the PEs using round-robin
- decouple the lattice subdivision used for the cell distribution from the one used for isosurfacing algorithm:
a finer lattice subdivision is used for the round-robin distribution scheme

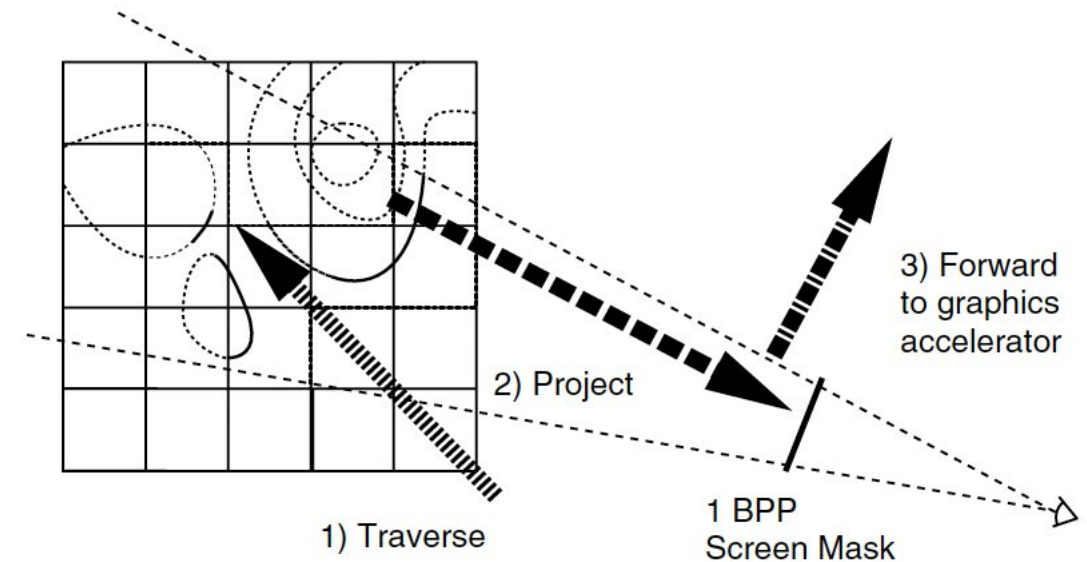




View dependent iso surface extraction



High depth complexity



3 steps algorithm